

Part 3: Implement a Change Impact Analyzer

-

Final Project CPSC 499.02: Software Analysis Winter 2026

Elda Britu - 30158734

Professor Robert Walker

April 13, 2026

To test the tool I wrote for this assignment, I created four sample Java methods (labeled as Subjects 1 through 4), each designed to highlight different types of dependencies. For each subject, I selected several change points and manually determined the expected impact sets. I then compared the tool's output to these expected results using precision, recall, and F1 score. The rest of this report explains how I chose the samples, how I performed the manual analysis, what results I got, and where the approach has limitations.

Sample Source Code and Change Point Selection

Rationale for Subject Selection

My goal was to cover the key dependency structures a PDG should capture, while keeping things simple enough to analyze by hand. Each subject increases slightly in complexity:

- **Subject1**, a simple sequential data chain with no branching or loops. This serves as a baseline where each statement depends on the previous one.
- **Subject2**, an if-else with two return statements. This introduces control dependence.
- **Subject3**, a while loop with loop-carried dependencies, making it the most structurally complex case.
- **Subject4**, a conditional structure with two independent variables, designed to test whether the tool avoids false positives.

Together, I believe these subjects cover data dependencies, control dependencies, loop back-edges, and independent execution paths, most of which were discussed in lectures. Note that the PDG includes some internal “block-opening” nodes that don't correspond to real lines of code. I will get back to this point later when discussing results.

Instead of testing every node, I chose change points that represent different roles in each method (e.g., source nodes, intermediate nodes, and leaf nodes). This gave me good coverage without unnecessary redundancy.

Manual Analysis and Expected Results

To determine the expected impact sets, I manually traced dependencies for each subject. My process was:

1. Draw the PDG by hand, identifying all data and control dependencies.
2. Perform a forward BFS starting from the change point.
3. Record all reachable nodes (excluding the starting node).
4. Double-check by thinking about what a developer would need to revisit after making that change.

I completed this analysis before looking at the tool's output to avoid bias.

Full Results Table

The full evaluation results are as shown in this screenshot. Perfect matches have $F1 = 1.0$, while deviations indicate mismatches between the tool and the expected results.

```
=== CIA Evaluation Report ===
```

Subject	Change	Reported	Expected	Precision	Recall	F1
Subject1	1	[2, 3, 4]	[2, 3, 4]	1.000	1.000	1.000
Subject1	2	[3, 4]	[3, 4]	1.000	1.000	1.000
Subject1	3	[4]	[4]	1.000	1.000	1.000
Subject1	4	[]	[]	1.000	1.000	1.000
Subject2	1	[2, 3]	[3, 4]	0.500	0.500	0.500
Subject2	3	[]	[]	1.000	1.000	1.000
Subject2	4	[]	[]	1.000	1.000	1.000
Subject3	1	[5, 7]	[5, 7]	1.000	1.000	1.000
Subject3	2	[3, 4, 5, 6, 7]	[3, 5, 6, 7]	0.800	1.000	0.889
Subject3	5	[7]	[7]	1.000	1.000	1.000
Subject3	6	[3, 4, 5, 7]	[3, 5, 7]	0.750	1.000	0.857
Subject4	1	[3, 4, 5, 6]	[3, 5, 6]	0.750	1.000	0.857
Subject4	2	[6]	[6]	1.000	1.000	1.000
Subject4	3	[4, 5, 6]	[5, 6]	0.667	1.000	0.800
Subject4	5	[6]	[6]	1.000	1.000	1.000

Across all 15 test cases:

- Mean precision: 0.898
- Mean recall: 0.967
- Mean F1: 0.927
- Perfect results: 10/15 (67%)
- Precision < 1.0: 5 cases
- Recall < 1.0: 1 case

Analysis

The Recall is Near-Perfect

In 14 of 15 entries recall is 1.0, meaning the tool never misses a truly impacted node. The single exception is Subject 2 change-at-1 (recall = 0.5), discussed below. This near-perfect recall is consistent with the theoretical guarantee of PDG-based CIA: PDGs over-approximate dependencies conservatively, so they should never miss a true dependence.

The only times precision falls below 1.0 were in four entries: Subject 3 change-at-2 (0.800), Subject 3 change-at-6 (0.750), Subject 4 change-at-1 (0.750), and Subject 4 change-at-3 (0.667). In every case the cause is the same: the StatementNodeBuilder inserts a structural "block" node at the start of each compound statement body. This block node is control-dependent on the enclosing predicate, so the forward BFS picks it up as an impacted node. Since the oracle excludes block nodes (they are not executable statements), the tool over-reports.

For example, in Subject 3 change-at-2 ($i = 0$), the reported set is [3, 4, 5, 6, 7] where node 4 is the while-body block node. The expected set is [3, 5, 6, 7]. Node 4 is a false positive produced by the PDG model including structural elements that have no executable meaning.

After reviewing the notes and looking through the code again, I believe this boils down to a genuine limitation of the mode. PDG-based CIA is not statement-level but node-level, and the granularity of the node model (which includes structural block nodes) determines what can and cannot be reported precisely.

The Subject 2 Anomaly

The most interesting entry is Subject 2 change-at-1, where both precision and recall are 0.5. The tool reported [2, 3] but the oracle expected [3, 4]. Node 2 is a false positive (the if-body block node), and node 4 is a false negative. The tool failed to report that the false-branch return statement (return $b - a$) is control-dependent on the if-predicate.

This is worth examining carefully. In Subject 2, the CFG has two branches from the predicate: the TRUE branch enters the if-body block (node 2) and eventually reaches the true-branch return (node 3); the FALSE branch goes directly to the false-branch return (node 4). In the post-dominator tree, neither node 3 nor node 4 post-dominates the predicate (node 1), so both should be control-dependent on it. However, the tool only reports node 2 and node 3, not node 4.

This suggests that the PDGBuilder's control-dependence computation is not connecting the predicate to the false-branch statement when the false branch goes directly to a return. The transitive closure added by PDGBuilder propagates control edges transitively through the block node ($1 \rightarrow 2 \rightarrow 3$) but does not independently produce the edge $1 \rightarrow 4$ for the direct false-branch path. This represents a genuine edge case in the PDG construction that causes a false negative; a situation where a real change impact is missed.

Overall Conclusion

Most of the imperfect results come from one issue: the PDGBuilder creates block-opening nodes for braces, as mentioned before. These nodes show up in the BFS results and don't correspond to actual code. As a result, it tended to reduce precision but didn't affect recall much, from what I could observe. So overall, the tool tends to over-approximate, which is generally acceptable for static analysis.

Otherwise, the tool is mostly sound (14 out of 15 cases), with the false positives being caused by implementation details that I do not doubt could be fixed (most likely by removing the block nodes). The Subject 2 anomaly reveals a more serious concern: a potential false negative caused by a gap in the PDGBuilder's control-dependence computation for direct false-branch paths. This is not a limitation of the PDG model in theory, but of the specific implementation. A developer relying on this tool in a real maintenance scenario would miss the impact on the false-branch return statement, which could lead to an incomplete regression test suite.

From these results, I think PDGs are a solid foundation for change impact analysis, but correctness depends heavily on how accurately the graph is built.

Threats to Validity

I acknowledge that this isn't a perfect solution for many reasons.

To start, **the sample size is small**. All four subjects are short methods with at most six executable statements. Real-world change impact problems involve hundreds of lines with complex control flow, method calls, exceptions, and aliasing. The conclusions drawn here may not generalise to larger programs. In particular, the precision loss from block nodes may become more or less pronounced in subjects with deeper nesting, and the false-negative bug observed in Subject 2 may or may not manifest in more complex branching structures.

I could have messed up the analysis. Manual analysis of even small programs is error-prone, particularly for loop-carried dependences (Subject 3) where it is easy to overlook that a reaching definition crosses a back-edge. I mitigated this by cross-checking against the --list output and tracing each chain step by step, but I cannot rule out errors in the oracle. If an oracle entry is wrong, a perfect tool result would appear as a false positive or false negative in the evaluation, distorting the precision and recall figures.

I designed the subjects specifically to test known properties of the PDG model: data chains, control dependence, loop back-edges, and variable independence. While this makes the evaluation informative, it also means **the subjects are not representative of arbitrary real-world code**. In particular, I avoided features such as method calls, arrays, exceptions, and aliasing, all of which are known to pose challenges for PDG-based analysis. The tool's performance on subjects with these features is unknown.

The oracle excludes block nodes on the grounds that they are not executable statements. However, **this boundary is somewhat arbitrary**. One could argue that the "block" node represents the entry into a compound statement and that its inclusion in the impact set is a reasonable conservative approximation. If the oracle had included block nodes, precision would be 1.0 for all entries. The evaluation conclusions depend on this definitional choice.

The evaluation is based on a single implementation of PDG construction (the one built in Part 2). If the PDGBuilder contains other bugs beyond the false-branch control-edge issue identified above, the evaluation conflates implementation errors with model limitations. To separate these concerns properly, the evaluation should be run against an independently implemented and verified PDG builder.

The oracle was derived by reasoning about data and control flow, **not by actually making changes to the subject programs and running their test suites to observe which tests break**. True change impact (in the sense of which tests are affected) may differ from dependence-based impact because not all dependences are exercised by tests, and some dependences may be guarded by conditions that never evaluate to the relevant branch at runtime.

A more rigorous evaluation would compare the tool's output against the set of tests that actually break when each change is applied.

References

“Compiler:Compile – Apache Maven Compiler Plugin.” Built by Maven. Accessed April 11, 2026.
<https://maven.apache.org/plugins/maven-compiler-plugin/compile-mojo.html#sourceDirectory>.

GeeksforGeeks. (2026, January 16). Breadth first search or BFS for a graph.
<https://www.geeksforgeeks.org/dsa/breadth-first-search-or-bfs-for-a-graph/>

“Guide to Working with Multiple Modules – Maven.” Built by Maven. Accessed April 11, 2026. <https://maven.apache.org/guides/mini/guide-multiple-modules.html>.

Java collections series | baeldung. (n.d.-a). <https://www.baeldung.com/java-collections>

Java with antlr | baeldung. (n.d.-b). <https://www.baeldung.com/java-antlr>

Overview. Accessed April 11, 2026. <https://docs.junit.org/6.0.3/overview.html>.

The program dependence graph and its use in optimization Jeanne Ferrante. (n.d.-c).
https://web.eecs.umich.edu/~mahlke/courses/583f23/reading/ferrante_toplas_87.pdf

P.S.

Thank you for your lessons this semester. I know I wasn't able to attend a majority of the lectures, let alone the final one, but you were someone that I enjoyed listening to. Your hands-on approach to going through the code helped a lot when it comes to actually justifying what I simply brushed off as common sense.

I will be graduating this spring(fingers crossed), so I'm grateful to have you as one of my final professors.

Have a great summer!